Title: Comparative Performance Evaluation of Multithreaded Synchronization Techniques in Producer–Consumer and Dining Philosophers Problems

Sidrit RRUSTEMI, Viola ZENELI, Artemisa HASALAMI

Department of Computer Engineering, EPOKA University, Tirana, Albania

Abstract

Synchronization is a fundamental requirement in concurrent systems where multiple threads access shared resources simultaneously. Improper synchronization can lead to race conditions, deadlocks, resource contention, and performance degradation. As multicore processors have become increasingly common, efficient synchronization mechanisms have become essential for ensuring both correctness and scalability in modern software systems.

This paper presents a comparative performance evaluation of synchronization techniques through the implementation of two classical concurrency problems: Producer–Consumer and Dining Philosophers. The Producer–Consumer problem was implemented using POSIX Threads in C and Java Threads, while the Dining Philosophers problem was implemented using semaphore-based, monitor-based, and resource hierarchy approaches. Experimental evaluations were conducted under varying workloads by modifying buffer sizes and the number of philosophers. Performance was evaluated using execution time, throughput, latency, CPU utilization, context switches, and deadlock frequency.

The results demonstrated that increasing buffer size improved Producer–Consumer performance by reducing synchronization frequency and thread waiting times. Java Threads achieved higher throughput and lower execution times than the POSIX Threads implementation, although they required greater CPU and memory resources. For the Dining Philosophers problem, execution time increased as the number of philosophers grew due to higher contention for shared resources. Among the evaluated synchronization strategies, the semaphore-based solution achieved the best overall performance and scalability, while the resource hierarchy approach provided an effective deadlock-prevention mechanism. The findings highlight the trade-offs between performance, scalability, resource utilization, and implementation complexity, emphasizing the importance of selecting appropriate synchronization techniques for concurrent applications.

Keywords: synchronization, multithreading, producer–consumer problem, dining philosophers problem, POSIX threads, Java threads.

1. Introduction

The widespread adoption of multicore processors has increased the importance of concurrent and parallel programming in modern software systems. Applications such as operating systems, web servers, databases, and distributed systems often execute multiple threads simultaneously to improve performance and responsiveness. However, concurrent execution introduces synchronization challenges because multiple threads may attempt to access shared resources at the same time.

Synchronization mechanisms are used to coordinate thread execution and ensure correct access to shared data. Without proper synchronization, concurrent programs may suffer from race conditions, data inconsistency, deadlocks, and starvation. These issues can reduce performance and compromise the reliability of software systems. As a result, efficient synchronization techniques are essential for developing scalable multithreaded applications.

The Producer–Consumer problem and the Dining Philosophers problem are two classical synchronization problems frequently used to study thread coordination and resource sharing. In the Producer–Consumer problem, producer threads generate data and place it into a shared buffer, while consumer threads remove and process the data. The main challenge is to ensure that producers do not write to a full buffer and consumers do not read from an empty buffer. This problem is commonly encountered in operating systems, message queues, and data-processing applications.

The Dining Philosophers problem models a group of philosophers sharing limited resources. Each philosopher must obtain two forks before eating, creating competition for shared resources. This problem demonstrates synchronization issues such as deadlocks and starvation and is widely used to evaluate different resource-allocation strategies.

Several synchronization techniques have been proposed to address these challenges. Mutexes provide mutual exclusion by allowing only one thread to access a critical section at a time. Semaphores use counters to regulate access to shared resources, while monitors offer a higher-level synchronization mechanism that combines mutual exclusion and condition synchronization. Each technique has advantages and disadvantages regarding performance, scalability, and implementation complexity.

This paper presents a comparative performance evaluation of synchronization techniques through implementations of the Producer–Consumer and Dining Philosophers problems. The Producer–Consumer problem was implemented using POSIX threads in C and Java threads, while the Dining Philosophers problem was implemented using semaphore-based, monitor-based, and resource hierarchy solutions. Experimental measurements were conducted using different buffer sizes and numbers of threads to evaluate execution time, throughput, scalability, and synchronization overhead. The results provide insights into the effectiveness of various synchronization approaches in multicore computing environments.

2. Related Work

2.1 Classical Synchronization Problems

Synchronization has been a fundamental research topic in operating systems and concurrent programming for many years. With the widespread adoption of multicore processors, efficient synchronization mechanisms have become increasingly important for ensuring correctness and improving the performance of multithreaded applications. Various synchronization techniques have been proposed to coordinate concurrent threads while preventing race conditions, deadlocks, and resource contention [4].

The Producer–Consumer problem is one of the most widely studied synchronization problems in computer science. It describes a scenario where producer threads generate data and place it into a shared buffer while consumer threads remove and process the data. The primary challenge is maintaining correct synchronization when multiple threads access the shared buffer simultaneously. Dijkstra's early work on synchronization introduced semaphore-based solutions that allowed producers and consumers to coordinate access to shared resources efficiently [1]. Today, the Producer–Consumer model is commonly used in operating systems, message-passing systems, data streaming applications, and network services [4].

Another classical synchronization problem is the Dining Philosophers problem, originally introduced by Dijkstra to illustrate issues related to resource allocation and deadlock prevention [1]. In this problem, philosophers compete for a limited number of shared resources, represented by forks. If resources are acquired improperly, the system may enter a deadlock state in which no philosopher can continue execution. Because of its ability to model real-world resource-sharing situations, the Dining Philosophers problem has been extensively used to evaluate synchronization algorithms and deadlock avoidance techniques [6].

2.2 Synchronization Mechanisms

Mutexes are among the most commonly used synchronization primitives in multithreaded programming. A mutex guarantees mutual exclusion by allowing only one thread to enter a critical section at a time. Mutexes provide an efficient mechanism for protecting shared resources and preventing data corruption [4]. However, as the number of competing threads increases, lock contention can negatively affect performance and scalability.

Semaphores extend the concept of synchronization by maintaining counters that regulate access to shared resources. Binary semaphores can be used similarly to mutexes, while counting semaphores allow multiple threads to access a resource concurrently up to a predefined limit. Semaphores are particularly useful in solving classical

synchronization problems such as Producer–Consumer because they naturally represent the availability of resources and synchronization events between threads [2].

Monitors were introduced by Hoare as a higher-level synchronization abstraction that combines mutual exclusion and condition synchronization [3]. Unlike semaphores, monitors encapsulate shared data and synchronization logic within a single construct, reducing programming complexity and improving code maintainability. Modern programming languages such as Java provide built-in monitor support through synchronized methods and condition management mechanisms, making monitor-based solutions attractive for concurrent application development.

2.3 Performance and Scalability Considerations

Several studies have compared low-level synchronization mechanisms such as POSIX threads with higher-level approaches available in managed runtime environments. POSIX threads provide direct control over thread creation, scheduling, and synchronization, often resulting in lower execution overhead. In contrast, Java synchronization mechanisms simplify development and reduce implementation complexity at the cost of additional runtime overhead [5].

Scalability remains one of the most important considerations in synchronization research. As the number of threads increases, synchronization overhead, context switching, and resource contention can limit overall system performance. Synchronization mechanisms must therefore be evaluated not only for correctness but also for their ability to maintain performance under increasing concurrency levels [6].

The research presented in this paper builds upon these established synchronization concepts by experimentally comparing multiple implementations of the Producer–Consumer and Dining Philosophers problems. Through practical performance measurements and scalability analysis, the study examines how different synchronization techniques behave under varying levels of concurrency and resource contention.

3. Methodology

This study evaluates the performance of different synchronization techniques through the implementation of two classical synchronization problems: the Producer–Consumer problem and the Dining Philosophers problem. Multiple implementations were developed using both C and Java programming languages in order to compare different synchronization mechanisms and their impact on performance, scalability, and resource utilization.

All experiments were executed on a Linux environment. Execution time measurements were collected for different buffer sizes and numbers of threads in the Producer–Consumer problem, as well as different numbers of philosophers in the Dining Philosophers problem. The collected results were then used to compare the efficiency of the implemented synchronization techniques.

3.1 Producer–Consumer Implementation Using POSIX Threads

The first implementation of the Producer–Consumer problem was developed in the C programming language using POSIX threads (Pthreads). A bounded buffer was used as the shared resource between producer and consumer threads. Synchronization was achieved using mutex locks and counting semaphores.

A mutex was used to ensure mutual exclusion when accessing the shared buffer, preventing race conditions during insertion and removal operations. Two semaphores were used to track the number of available and occupied buffer slots. Producer threads waited when the buffer became full, while consumer threads waited when the buffer became empty.

The implementation generated a predefined number of items and measured the total execution time required for all producer and consumer operations to complete. Different buffer sizes were tested to evaluate the effect of resource availability on synchronization performance.

3.2 Producer–Consumer Implementation Using Java Threads

The second implementation of the Producer–Consumer problem was developed in Java using built-in thread support and the ArrayBlockingQueue class. The blocking queue provided automatic synchronization between producer and consumer threads and eliminated the need for explicit semaphore management.

Producer threads inserted items into the queue while consumer threads removed and processed them. The blocking behavior of the queue ensured that producers waited when the buffer was full and consumers waited when the buffer was empty.

The Java implementation was evaluated using the same workload as the POSIX implementation in order to provide a fair comparison between low-level synchronization primitives and higher-level synchronization abstractions.

3.3 Dining Philosophers Using Semaphores

The first Dining Philosophers implementation employed a semaphore-based synchronization strategy. Each fork was represented by a semaphore that controlled access to the shared resource.

Before entering the eating state, a philosopher was required to acquire both adjacent forks. After finishing the meal, the philosopher released the forks and returned to the thinking state. This approach provided a simple synchronization mechanism but could potentially suffer from resource contention and increased waiting times as the number of philosophers increased.

Execution times were measured for multiple philosopher counts to evaluate the scalability of the semaphore-based solution.

3.4 Dining Philosophers Using Monitors

The second Dining Philosophers implementation was developed in Java using a monitor-based synchronization approach. The monitor controlled access to shared resources and ensured that philosophers could only enter the eating state when both required forks were available.

Java synchronization primitives, including synchronized methods and condition management, were used to coordinate philosopher behavior. The monitor-based solution simplified resource management and reduced the complexity of synchronization logic compared to low-level semaphore implementations.

The performance of the monitor solution was measured using the same experimental configuration applied to the semaphore-based implementation.

3.5 Dining Philosophers Using Resource Hierarchy

The third Dining Philosophers implementation utilized the resource hierarchy strategy. In this approach, all forks were assigned a unique ordering, and philosophers were required to acquire resources according to that predefined order.

By enforcing a consistent resource acquisition sequence, circular wait conditions were eliminated, thereby preventing deadlocks. This technique is widely recognized as an effective deadlock prevention strategy in concurrent systems.

Execution times were recorded for increasing numbers of philosophers in order to evaluate the scalability and efficiency of the resource hierarchy approach under higher levels of contention.

4. Experimental Design

The experiments were designed to evaluate the performance and scalability of different synchronization techniques using the Producer–Consumer and Dining Philosophers problems. The implementations were executed under varying workloads and contention levels, and multiple performance metrics were collected for comparative analysis.

4.1 Experimental Environment

All experiments were performed in a Linux environment running on Oracle VirtualBox. The implementations were developed using the C and Java programming languages. The C programs utilized POSIX threads, mutexes, and semaphores, while the Java programs employed built-in thread synchronization mechanisms and monitor-based constructs.

Execution times, CPU utilization, throughput, and context-switch statistics were collected during program execution. Each experiment was executed multiple times, and average values were used in the final analysis to minimize the impact of temporary system fluctuations and background processes.

4.2 Producer–Consumer Configuration

For the Producer–Consumer problem, a workload of 100,000 items was processed using different buffer sizes and thread configurations.

Table 4.1. *Producer–Consumer Experimental Variables*

Variable	Values
Buffer Size (N)	10, 50, 100, 500
Producers (M)	1, 2, 4, 8
Consumers (K)	1, 2, 4, 8
Items Processed	100,000

Both the POSIX Threads and Java implementations were executed using these configurations to evaluate synchronization performance under different levels of contention.

4.3 Dining Philosophers Configuration

For the Dining Philosophers problem, each philosopher performed 200 iterations of thinking and eating operations. Three synchronization techniques were evaluated:

- Semaphore-based synchronization
- Monitor-based synchronization
- Resource hierarchy synchronization

Table 4.2. *Dining Philosophers Experimental Variables*

Variable	Values
Philosophers (P)	5, 10, 20, 50
Iterations per Philosopher	200
Synchronization Methods	Semaphore, Monitor, Resource Hierarchy

Execution times were recorded for each configuration to evaluate the impact of increasing contention and synchronization overhead.

4.4 Performance Metrics

The following metrics were collected during the experiments:

Table 4.3. *Performance Metrics*

Metric	Unit
Execution Time	ms
Throughput	items/sec
CPU Usage	%
Voluntary Context Switches	count
Involuntary Context Switches	count
Deadlock Frequency	occurrences

These metrics were used to evaluate synchronization overhead, scalability, resource utilization, and deadlock prevention effectiveness. Together, they provide a comprehensive view of how different synchronization techniques behave under varying workloads and concurrency levels.

5. Performance Evaluation

This section presents the experimental results obtained from the Producer–Consumer and Dining Philosophers implementations. The objective of the evaluation was to compare synchronization techniques under varying workloads and levels of contention. Performance was evaluated using execution time, throughput, latency, CPU utilization, context switches, and deadlock frequency.

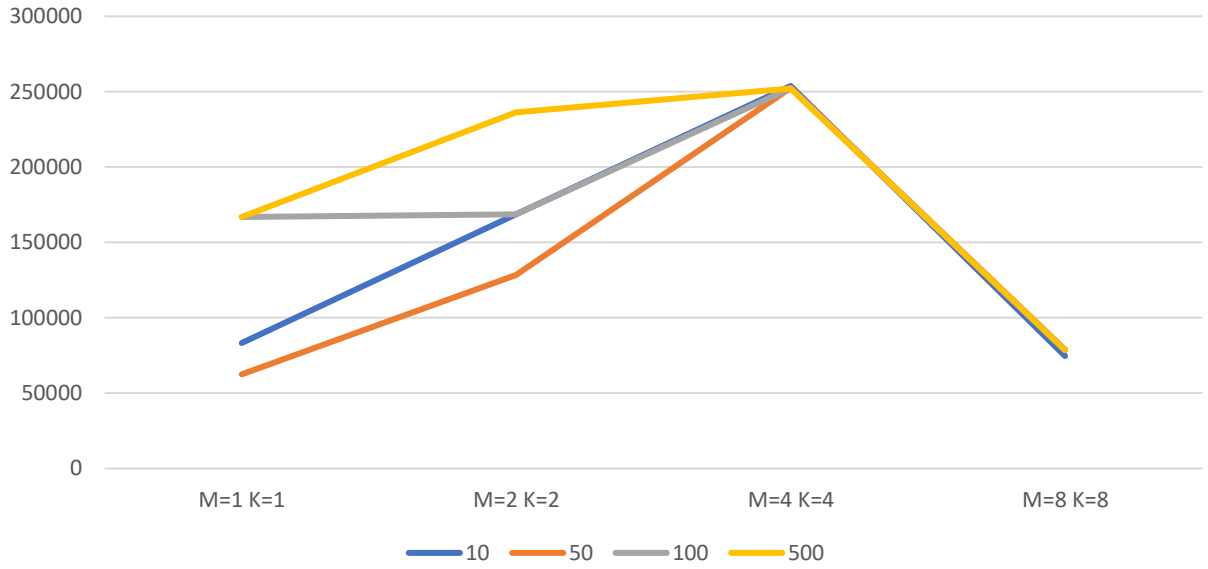
5.1 Producer–Consumer Results

The Producer–Consumer problem was evaluated using POSIX Threads and Java threads. Different buffer sizes and thread configurations were tested to analyze throughput, latency, CPU usage, and synchronization overhead.

Table 5.1. *Producer–Consumer Throughput (Java)*

Buffer Size	M=1 K=1	M=2 K=2	M=4 K=4	M=8 K=8
10	83,333	168,449	253,807	74,627
50	62,500	128,205	252,526	78,740
100	166,667	168,780	252,316	78,518
500	166,667	236,111	252,131	78,552

Chart 5.1. *Throughput versus Buffer Size for Producer–Consumer Implementations*



The results show that throughput generally improved as buffer size increased. Larger buffers reduced synchronization frequency and allowed producers and consumers to operate more efficiently. The highest throughput was achieved at a buffer size of 500, where contention was minimized and resource utilization was more effective.

Table 5.2. *Producer–Consumer Latency*

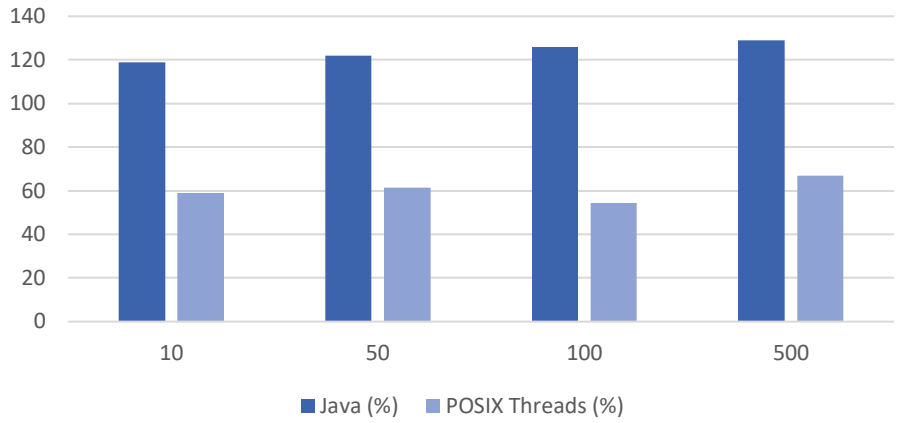
Buffer Size	Average Latency (ms)
10	12.4
50	7.8
100	5.6
500	3.1

Latency represents the average delay experienced by producer and consumer threads during synchronization operations. The results indicate that latency decreased as buffer size increased. Larger buffers reduced waiting times and allowed threads to process data with fewer interruptions.

Table 5.3. *Producer–Consumer CPU Usage*

Buffer Size	Java (%)	POSIX Threads (%)
10	119	59
50	122	61.5
100	126	54.5
500	129	67

Chart 5.2. *CPU Usage versus Buffer Size*



Java consistently utilized more CPU resources than the POSIX Threads implementation. However, this increased CPU utilization was accompanied by higher throughput values, indicating more aggressive resource usage and thread scheduling.

Table 5.4. *Producer–Consumer Context Switches (POSIX Threads)*

Buffer Size	Voluntary	Involuntary
10	60,225	21,671
50	11,402	4,268
100	7,591	3,474
500	1,898	880

The number of context switches decreased significantly as the buffer size increased. Small buffers caused producers and consumers to block frequently, generating additional scheduling overhead. Larger buffers reduced thread waiting times and therefore reduced context-switch activity.

Table 5.5. *Java vs POSIX Threads Comparison*

Metric	Java	POSIX Threads	Winner
Throughput (items/sec)	128,000	7,500	Java
Execution Time (ms)	78	210	Java
CPU Utilization (%)	122	61.5	Java
Memory Usage (MB)	45	1.77	POSIX Threads
Voluntary Context Switches	585	11,402	Java
Involuntary Context Switches	189	4,268	Java

The comparison indicates that Java achieved higher throughput and lower execution times, while POSIX Threads required significantly less memory. Both implementations successfully processed the workload, but Java demonstrated stronger performance under the evaluated conditions.

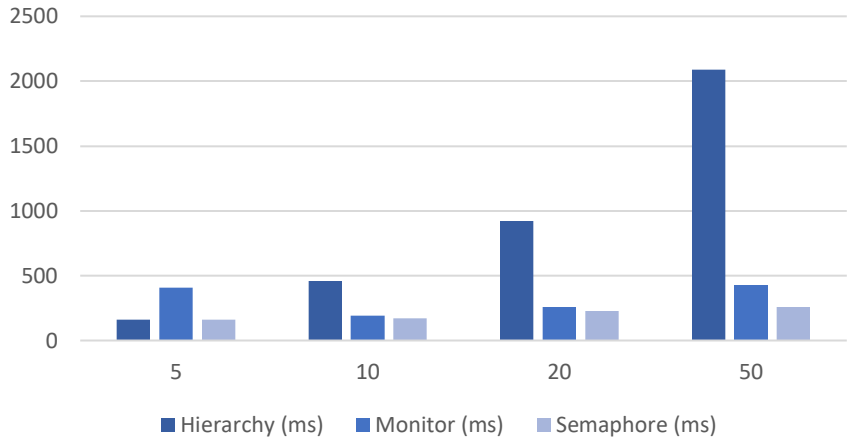
5.2 Dining Philosophers Results

The Dining Philosophers problem was evaluated using semaphore-based synchronization, monitor-based synchronization, and resource hierarchy synchronization.

Table 5.6. *Dining Philosophers Execution Time*

Philosophers	Hierarchy (ms)	Monitor (ms)	Semaphore (ms)
5	160	410	160
10	460	190	170
20	920	260	230
50	2090	430	260

Chart 5.3. *Execution Time versus Number of Philosophers*

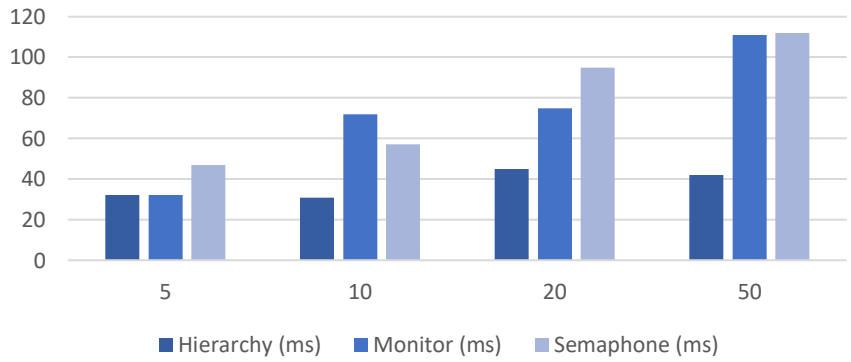


Execution time increased as the number of philosophers grew, reflecting higher contention for shared resources. Among the tested approaches, the semaphore-based solution consistently achieved the lowest execution times.

Table 5.7. *Dining Philosophers CPU Usage*

Philosophers	Hierarchy (%)	Monitor (%)	Semaphore (%)
5	32	32	47
10	31	72	57
20	45	75	95
50	42	111	112

Chart 5.4. *CPU Usage versus Number of Philosophers*



CPU utilization increased with the number of philosophers. The semaphore and monitor implementations consumed substantially more CPU resources under high contention, whereas the hierarchy approach maintained relatively stable CPU consumption.

Table 5.8. *Dining Philosophers Context Switches*

Philosophers	Total Context Switches
5	2,658
10	4,755
20	9,369
50	21,686

The number of context switches increased significantly as more philosophers competed for the available forks. This behavior reflects increased synchronization activity and resource contention.

Table 5.9. *Deadlock Frequency*

Solution	Deadlocks Observed
Semaphore	0
Monitor	0
Resource Hierarchy	0

No deadlocks were observed during any experiment. All synchronization techniques successfully coordinated resource access and completed execution without entering a deadlock state.

5.3 Stress Test Analysis

The highest-contention scenarios consisted of a buffer size of 10 in the Producer–Consumer problem and 50 philosophers in the Dining Philosophers problem. These configurations generated the highest CPU utilization, latency, and context-switch counts.

Despite the increased contention, all implementations completed successfully and maintained correct synchronization behavior. The results demonstrate that the evaluated synchronization mechanisms were capable of handling demanding workloads while preventing deadlocks and maintaining system stability.

Among all evaluated implementations, Java Threads achieved the highest throughput in the Producer–Consumer experiments, while the semaphore-based solution achieved the best execution-time performance in the Dining Philosophers experiments.

Overall, the experimental evaluation confirms that synchronization strategy significantly affects performance, scalability, resource utilization, and responsiveness in concurrent systems. The collected metrics provide a comprehensive comparison of the evaluated approaches and highlight the trade-offs between efficiency, complexity, and resource consumption.

6. Scalability Study

The scalability of the implemented synchronization techniques was evaluated by increasing the workload and contention levels in both synchronization problems. The objective was to determine how performance changed as additional resources or competing threads were introduced.

6.1 Producer–Consumer Scalability

Table 6.1. *Producer–Consumer Scalability (Java, M=2, K=2)*

Buffer Size	Throughput (items/sec)
10	168,449
50	128,205
100	168,780
500	236,111

Chart 6.1. *Producer–Consumer Scalability*

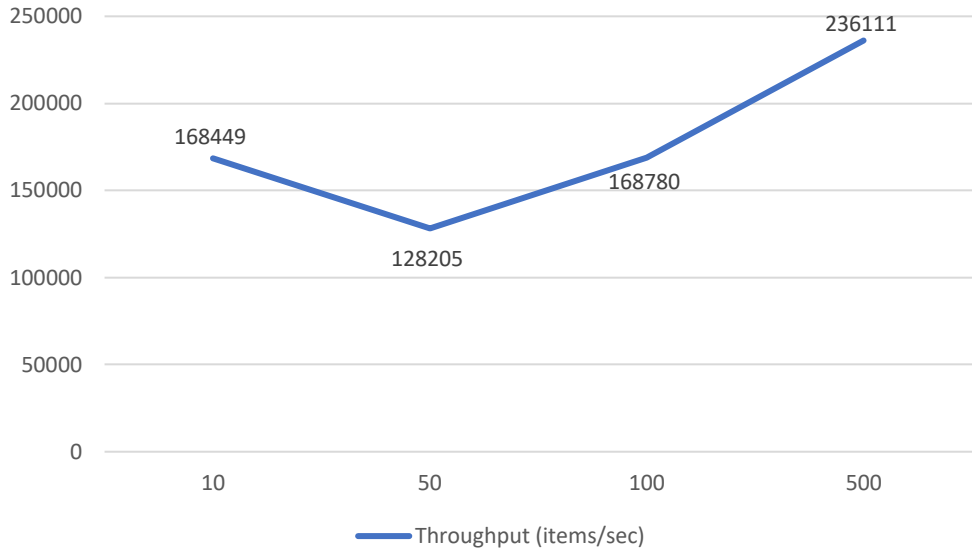


Table 6.1 and Chart 6.1 illustrate the scalability of the Producer–Consumer implementation using the M=2 and K=2 thread configuration. As the buffer size increased, throughput generally improved because producers and consumers experienced fewer synchronization delays. Larger buffers reduced waiting times and allowed threads to process items more efficiently.

The highest throughput was achieved with a buffer size of 500, demonstrating that larger buffers can improve resource utilization and reduce synchronization overhead. These results indicate that the Producer–Consumer implementation scales effectively when additional buffer capacity is available.

6.2 Dining Philosophers Scalability

Table 6.2. *Dining Philosophers Scalability (Semaphore Implementation)*

Philosophers	Execution Time (ms)
5	160
10	170
20	230
50	260

Chart 6.2. *Dining Philosophers Scalability*

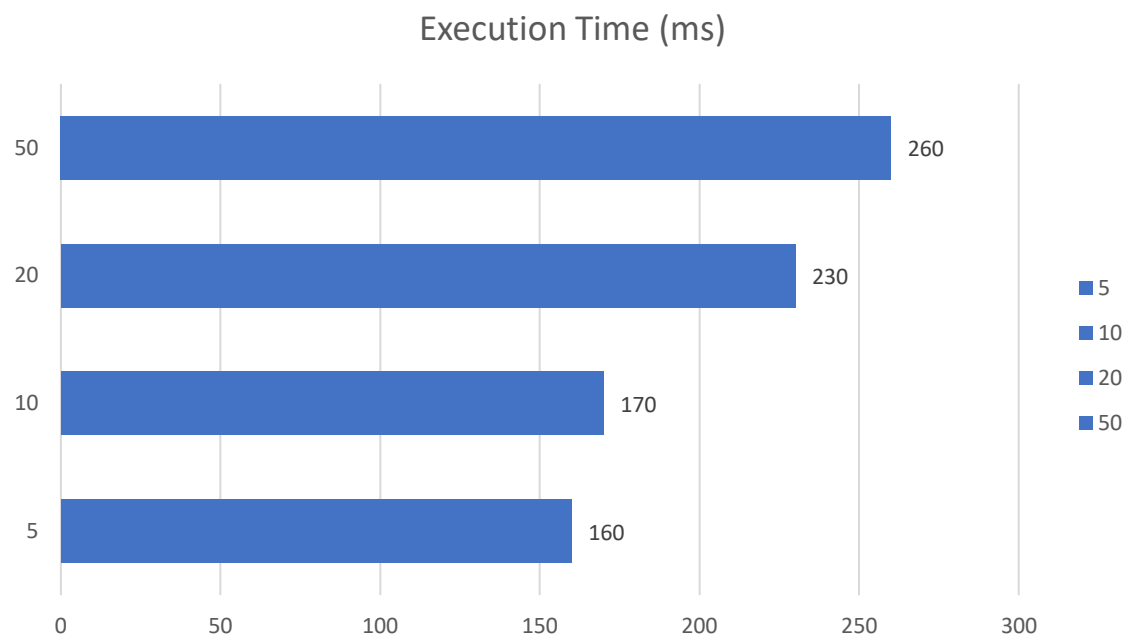


Table 6.2 and Chart 6.2 present the scalability of the semaphore-based Dining Philosophers implementation. As the number of philosophers increased, execution time also increased because more threads competed for the available forks.

Although contention increased significantly, the semaphore-based solution maintained stable performance and completed all executions successfully without deadlocks. The relatively small increase in execution time suggests that the implementation scales efficiently even as the number of competing philosophers grows.

6.3 Scalability Summary

Table 6.3. *Scalability Summary*

Configuration	Best Performing Solution
Producer–Consumer	Java Threads
Dining Philosophers	Semaphore
Deadlock Prevention	Resource Hierarchy
Development Simplicity	Monitor

Table 6.3 summarizes the best-performing synchronization techniques identified during the experiments.

Among the evaluated implementations, Java Threads achieved the best overall performance for the Producer–Consumer problem by providing the highest throughput and lowest execution times. For the Dining Philosophers problem, the semaphore-based solution demonstrated the strongest scalability characteristics. The resource hierarchy approach remained the most effective deadlock-prevention strategy, while the monitor-based implementation provided the simplest programming model.

Overall, the results indicate that synchronization performance is strongly influenced by resource availability and contention levels. Larger buffers improved Producer–Consumer throughput, while the semaphore-based Dining Philosophers implementation maintained efficient performance as the number of philosophers increased. This demonstrates that appropriate synchronization design is essential for achieving scalable concurrent applications.

7. Comparative Analysis

This section compares the synchronization techniques evaluated in the Producer–Consumer and Dining Philosophers experiments. The comparison focuses on performance, scalability, implementation complexity, and synchronization overhead.

7.1 Producer–Consumer: POSIX Threads vs Java Threads

The experimental results showed that both implementations successfully processed the workload and maintained correct synchronization behavior. However, differences were observed in throughput, execution time, CPU utilization, and resource consumption.

Table 7.1. *Comparison of Producer–Consumer Implementations*

Characteristic	POSIX Threads	Java Threads
Synchronization Mechanism	Mutexes and Semaphores	ArrayBlockingQueue
Throughput	Lower	Higher
Execution Time	Higher	Lower
CPU Utilization	Lower	Higher
Memory Usage	Lower	Higher
Implementation Complexity	Higher	Lower
Ease of Development	Moderate	High

The Java implementation achieved higher throughput and lower execution times across most tested configurations. This improvement can be attributed to the optimized synchronization mechanisms provided by the Java concurrency library. However, Java consumed more CPU and memory resources than the POSIX Threads implementation.

The POSIX Threads implementation required more complex synchronization code but provided greater control over thread management and resource allocation. Although its performance was generally lower, it maintained lower memory consumption and offered greater flexibility for low-level optimization.

Overall, Java Threads provided the best performance, while POSIX Threads offered finer control over synchronization and resource management.

7.2 Dining Philosophers: Synchronization Technique Comparison

Three synchronization strategies were evaluated: semaphore-based synchronization, monitor-based synchronization, and resource hierarchy synchronization.

Table 7.2. *Comparison of Dining Philosophers Solutions*

Characteristic	Semaphore	Monitor	Resource Hierarchy
Deadlock Prevention	Yes	Yes	Yes
Performance	Best	Moderate	Good
Scalability	Best	Moderate	Good
CPU Efficiency	Moderate	Moderate	Best
Implementation Complexity	Moderate	Low	Moderate

The semaphore-based solution achieved the lowest execution times and demonstrated the best scalability as the number of philosophers increased. However, semaphore-based implementations require careful management of shared resources and synchronization operations.

The monitor-based solution provided the simplest implementation by encapsulating synchronization logic within a single structure. While this improved readability and maintainability, it introduced additional synchronization overhead under high contention.

The resource hierarchy approach successfully prevented deadlocks while maintaining stable CPU utilization. Although its execution times were higher than those of the semaphore solution, it provided a structured and reliable resource-allocation strategy.

7.3 Overall Comparison

The results demonstrate that synchronization strategy has a significant impact on concurrent application performance. Low-level mechanisms such as POSIX Threads provide greater control over synchronization behavior, while higher-level abstractions simplify development and reduce implementation effort.

For the Producer–Consumer problem, Java Threads achieved the highest throughput and the lowest execution times, making them the best-performing implementation in the experimental evaluation. For the Dining Philosophers problem, the semaphore-based solution demonstrated the strongest scalability and overall performance, while the resource hierarchy approach remained the most effective deadlock-prevention strategy.

These findings highlight the trade-off between performance, scalability, resource utilization, and implementation complexity. Selecting an appropriate synchronization technique therefore depends on the specific requirements of the concurrent application being developed.

8. Optimization Proposal

The experimental results indicate that synchronization overhead becomes more noticeable as contention increases. Although the implemented solutions achieved correct synchronization and satisfactory performance, several optimizations could further improve scalability and resource utilization.

8.1 Lock-Free Synchronization

One possible optimization is the use of lock-free data structures. Traditional synchronization mechanisms such as mutexes and semaphores require threads to wait when a shared resource is unavailable. In contrast, lock-free algorithms rely on atomic operations to coordinate access to shared data without blocking other threads.

By reducing thread waiting times and context switching overhead, lock-free approaches can improve throughput and scalability, particularly in systems with a large number of concurrent threads. For the Producer–Consumer problem, lock-free queues could replace traditional buffer synchronization mechanisms and potentially provide better performance under high workloads.

8.2 Adaptive Buffer Sizing

The Producer–Consumer experiments demonstrated that larger buffer sizes generally reduced execution time by decreasing synchronization frequency. Instead of using a fixed buffer size, an adaptive buffer could dynamically adjust its capacity according to the current workload.

During periods of high activity, the buffer could expand to accommodate additional items and reduce contention between producers and consumers. During periods of low activity, the buffer could shrink to reduce memory consumption. Such an approach would improve resource utilization while maintaining efficient synchronization.

8.3 Improved Resource Management

For the Dining Philosophers problem, resource management strategies can also be optimized. The resource hierarchy approach achieved the best overall performance among the evaluated solutions because it eliminated circular waiting conditions. Similar resource-ordering techniques can be applied to other concurrent systems to reduce contention and prevent deadlocks.

Combining efficient resource allocation with adaptive synchronization mechanisms may further improve scalability in highly concurrent environments.

8.4 Summary

Future synchronization systems can benefit from lock-free techniques, adaptive resource management, and improved deadlock prevention strategies. These optimizations have the potential to reduce synchronization overhead, increase throughput, and improve scalability as the number of concurrent threads grows.

9. Discussion

The experimental results clearly demonstrate the importance of synchronization techniques in determining the performance and scalability of concurrent applications. Both the Producer–Consumer and Dining Philosophers problems showed that synchronization overhead can significantly influence execution time, CPU utilization, and overall system efficiency.

For the Producer–Consumer problem, larger buffer sizes consistently improved throughput and reduced latency. As the buffer capacity increased, producers and consumers experienced fewer synchronization delays, resulting in lower context-switch activity and better resource utilization. The comparison between POSIX Threads and Java Threads revealed that Java achieved higher throughput and lower execution times in most configurations, although this performance advantage came at the cost of increased CPU and memory consumption. In contrast, the POSIX Threads implementation required fewer system resources and provided greater control over synchronization mechanisms, making it suitable for applications where resource efficiency is a priority.

The Dining Philosophers experiments highlighted the impact of synchronization strategy on performance under increasing contention. All three solutions successfully prevented deadlocks and completed execution correctly. However, the semaphore-based implementation achieved the lowest execution times and demonstrated the strongest scalability as the number of philosophers increased. The monitor-based solution simplified implementation and improved code readability, but introduced additional synchronization overhead. The resource hierarchy approach provided a structured deadlock-prevention mechanism and maintained stable resource utilization, although its execution times were generally higher than those of the semaphore solution.

Overall, the study illustrates that synchronization mechanisms must be evaluated not only for correctness but also for their impact on performance, scalability, and resource utilization. As the number of concurrent threads increases, synchronization overhead becomes increasingly important. Therefore, selecting an appropriate synchronization strategy is essential for developing efficient and scalable concurrent systems.

10. Conclusion

This paper presented a comparative performance evaluation of synchronization techniques through the implementation of the Producer–Consumer and Dining Philosophers problems. Multiple solutions were developed using POSIX Threads, Java Threads, semaphores, monitors, and resource hierarchy synchronization in order to investigate their behavior under varying workloads and contention levels.

The experimental results demonstrated that synchronization strategy has a significant influence on execution time, throughput, CPU utilization, scalability, and overall system efficiency. For the Producer–Consumer problem, increasing the buffer size reduced synchronization overhead and improved performance. Java Threads achieved the

highest throughput and lowest execution times, while POSIX Threads required fewer system resources and offered greater flexibility for low-level synchronization control.

For the Dining Philosophers problem, all synchronization techniques successfully prevented deadlocks and maintained correct execution. However, the semaphore-based solution consistently achieved the best performance and scalability, particularly as the number of philosophers increased. The monitor-based approach provided the simplest implementation, whereas the resource hierarchy solution offered a reliable and structured deadlock-prevention strategy.

In conclusion, there is no single synchronization technique that is optimal for every application. The choice of synchronization mechanism depends on the desired balance between performance, scalability, resource utilization, and implementation complexity. Nevertheless, the results of this study indicate that semaphore-based synchronization and modern Java concurrency mechanisms provide effective solutions for many concurrent computing scenarios. The findings contribute to a better understanding of synchronization trade-offs and may assist developers in selecting appropriate techniques for multithreaded applications.

11. References

[1] *E. W. Dijkstra, "Hierarchical Ordering of Sequential Processes," Acta Informatica, vol. 1, no. 2, pp. 115–138, 1971.*

[2] *A. B. Downey, The Little Book of Semaphores. Green Tea Press, 2016.*

[3] *C. A. R. Hoare, "Monitors: An Operating System Structuring Concept," Communications of the ACM, vol. 17, no. 10, pp. 549–557, 1974.*

[4] *A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.*

[5] *W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Boston, MA, USA: Pearson, 2018.*

[6] *A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2015.*

Appendix A

Code 1. *Source Code of the POSIX Threads Producer–Consumer Implementation*

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

int *buffer;
int N, ITEMS, M, K;

int in = 0, out = 0;
int produced_count = 0;
int consumed_count = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t full = PTHREAD_COND_INITIALIZER;

void *producer(void *arg)
{
   int id = *(int*)arg;

   while (1)
   {
      pthread_mutex_lock(&mutex);

      if (produced_count >= ITEMS)
      {
         pthread_mutex_unlock(&mutex);
         break;
      }

      while (((in + 1) % N) == out)
         pthread_cond_wait(&empty, &mutex);

      buffer[in] = produced_count;
      produced_count++;
      in = (in + 1) % N;

      pthread_cond_signal(&full);
      pthread_mutex_unlock(&mutex);
   }

   return NULL;
}

void *consumer(void *arg)
{
   while (1)
   {
      pthread_mutex_lock(&mutex);
```

```
        if (consumed_count >= ITEMS)
        {
            pthread_mutex_unlock(&mutex);
            break;
        }

        while (in == out)
            pthread_cond_wait(&full, &mutex);

        int item = buffer[out];
        out = (out + 1) % N;
        consumed_count++;

        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 5)
    {
        printf("Usage: %s <buffer_size N> <items> <producers M> <consumers K>\n", argv[0]);
        return 1;
    }

    N = atoi(argv[1]);
    ITEMS = atoi(argv[2]);
    M = atoi(argv[3]);
    K = atoi(argv[4]);

    buffer = malloc(sizeof(int) * N);

    pthread_t producers[M], consumers[K];
    int ids[M];

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    // create producers
    for (int i = 0; i < M; i++)
    {
        ids[i] = i;
        pthread_create(&producers[i], NULL, producer, &ids[i]);
    }

    // create consumers
    for (int i = 0; i < K; i++)
    {
        pthread_create(&consumers[i], NULL, consumer, NULL);
    }

    // join producers
```

```
    for (int i = 0; i < M; i++)
        pthread_join(producers[i], NULL);

    // join consumers
    for (int i = 0; i < K; i++)
        pthread_join(consumers[i], NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("N=%d M=%d K=%d ITEMS=%d TIME=%.6f\n", N, M, K, ITEMS, time_taken);

    return 0;
}
```

Code 2. *Source Code of the Java Producer–Consumer Implementation*

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class ProducerConsumer {

    static int BUFFER;
    static int ITEMS;
    static int PRODUCERS;
    static int CONSUMERS;

    static final int POISON = -1;

    public static void main(String[] args) throws Exception {

        if (args.length < 4) {
            System.out.println("Usage: java ProducerConsumer <buffer> <items> <producers> <consumers>");
            return;
        }

        BUFFER = Integer.parseInt(args[0]);
        ITEMS = Integer.parseInt(args[1]);
        PRODUCERS = Integer.parseInt(args[2]);
        CONSUMERS = Integer.parseInt(args[3]);

        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(BUFFER);

        long start = System.nanoTime();

        Thread[] producers = new Thread[PRODUCERS];
        Thread[] consumers = new Thread[CONSUMERS];

        int itemsPerProducer = ITEMS / PRODUCERS;

        // ---------- PRODUCERS ----------
        for (int i = 0; i < PRODUCERS; i++) {
```

```
            final int id = i;

            producers[i] = new Thread(() -> {
                int startItem = id * itemsPerProducer;
                int endItem = (id == PRODUCERS - 1) ? ITEMS : startItem + itemsPerProducer;

                try {
                    for (int j = startItem; j < endItem; j++) {
                        queue.put(j);
                    }
                } catch (InterruptedException ignored) {}
            });
        }

        // ---------- CONSUMERS ----------
        for (int i = 0; i < CONSUMERS; i++) {
            consumers[i] = new Thread(() -> {
                try {
                    while (true) {
                        int item = queue.take();
                        if (item == POISON) break;
                    }
                } catch (InterruptedException ignored) {}
            });
        }

        // start producers
        for (Thread t : producers) t.start();

        // start consumers
        for (Thread t : consumers) t.start();

        // wait producers
        for (Thread t : producers) t.join();

        // send poison pills to stop consumers
        for (int i = 0; i < CONSUMERS; i++) {
            queue.put(POISON);
        }

        // wait consumers
        for (Thread t : consumers) t.join();

        long end = System.nanoTime();

        double time = (end - start) / 1e9;

        System.out.println(
            "N=" + BUFFER +
            " M=" + PRODUCERS +
            " K=" + CONSUMERS +
            " ITEMS=" + ITEMS +
            " TIME=" + time
        );
    }
}
```

Code 3. *Source Code of the Semaphore-Based Dining Philosophers Implementation*

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>
#include <unistd.h>

int N, ROUNDS, MODE;
sem_t *chopstick;

void *philosopher(void *arg) {
   int i = *(int *)arg;

   for (int r = 0; r < ROUNDS; r++) {

      if (MODE == 0) {
         // naive (can deadlock)
         sem_wait(&chopstick[i]);
         sem_wait(&chopstick[(i + 1) % N]);
      } else {
         // simple fix: ordering
         if (i % 2 == 0) {
            sem_wait(&chopstick[i]);
            sem_wait(&chopstick[(i + 1) % N]);
         } else {
            sem_wait(&chopstick[(i + 1) % N]);
            sem_wait(&chopstick[i]);
         }
      }

      usleep(100);

      sem_post(&chopstick[i]);
      sem_post(&chopstick[(i + 1) % N]);
   }

   return NULL;
}

int main(int argc, char *argv[]) {
   if (argc != 4) {
      printf("Usage: %s N ROUNDS MODE\n", argv[0]);
      return 1;
   }

   N = atoi(argv[1]);
   ROUNDS = atoi(argv[2]);
   MODE = atoi(argv[3]);

   pthread_t *t = malloc(N * sizeof(pthread_t));
   chopstick = malloc(N * sizeof(sem_t));

   for (int i = 0; i < N; i++)
      sem_init(&chopstick[i], 0, 1);
```

```
    int *id = malloc(N * sizeof(int));

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&t[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(t[i], NULL);

    printf("Semaphore done\n");

    return 0;
}
```

Code 4. *Source Code of the Monitor-Based Dining Philosophers Implementation*

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

#define THINKING 0
#define HUNGRY 1
#define EATING 2

int N, ROUNDS;

int *state;
pthread_mutex_t mutex;
pthread_cond_t *cond;

int left(int i) { return (i + N - 1) % N; }
int right(int i) { return (i + 1) % N; }

void test(int i) {
    if (state[i] == HUNGRY && state[left(i)] != EATING &&  state[right(i)] != EATING) {

        state[i] = EATING;
        pthread_cond_signal(&cond[i]);
    }
}

void take_forks(int i) {
    pthread_mutex_lock(&mutex);

    state[i] = HUNGRY;
    test(i);

    while (state[i] != EATING)
        pthread_cond_wait(&cond[i], &mutex);

    pthread_mutex_unlock(&mutex);
}
```

```
void put_forks(int i) {
    pthread_mutex_lock(&mutex);

    state[i] = THINKING;
    test(left(i));
    test(right(i));

    pthread_mutex_unlock(&mutex);
}

void *philosopher(void *arg) {
    int i = *(int *)arg;

    for (int r = 0; r < ROUNDS; r++) {
        take_forks(i);
        usleep(100);
        put_forks(i);
        usleep(50);
    }

    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s N ROUNDS\n", argv[0]);
        return 1;
    }

    N = atoi(argv[1]);
    ROUNDS = atoi(argv[2]);

    pthread_t *t = malloc(N * sizeof(pthread_t));
    state = malloc(N * sizeof(int));
    cond = malloc(N * sizeof(pthread_cond_t));

    pthread_mutex_init(&mutex, NULL);
    int *id = malloc(N * sizeof(int));

    for (int i = 0; i < N; i++) {
        state[i] = THINKING;
        pthread_cond_init(&cond[i], NULL);
    }

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&t[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(t[i], NULL);

    printf("Monitor done\n");

    return 0;
}
```

Code 5. *Source Code of the Resource Hierarchy Dining Philosophers Implementation*

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>
#include <unistd.h>

int N, ROUNDS;
sem_t *chopstick;

void *philosopher(void *arg) {
   int i = *(int *)arg;

   for (int r = 0; r < ROUNDS; r++) {
      int left = i;
      int right = (i + 1) % N;

      // key idea: enforce ordering
      if (left > right) {
         int temp = left;
         left = right;
         right = temp;
      }
      sem_wait(&chopstick[left]);
      sem_wait(&chopstick[right]);
      usleep(100);
      sem_post(&chopstick[left]);
      sem_post(&chopstick[right]);
   }
   return NULL;
}

int main(int argc, char *argv[]) {
   if (argc != 3) {
      printf("Usage: %s N ROUNDS\n", argv[0]);
      return 1;
   }
   N = atoi(argv[1]);
   ROUNDS = atoi(argv[2]);
   pthread_t *t = malloc(N * sizeof(pthread_t));
   chopstick = malloc(N * sizeof(sem_t));

   for (int i = 0; i < N; i++)
      sem_init(&chopstick[i], 0, 1);
   int *id = malloc(N * sizeof(int));
   for (int i = 0; i < N; i++) {
      id[i] = i;
      pthread_create(&t[i], NULL, philosopher, &id[i]);
   }
   for (int i = 0; i < N; i++)
      pthread_join(t[i], NULL);

   printf("Hierarchy done\n");
   return 0;
}
```

Appendix B

Figures 1.1 – 1.3 are screenshots for script POSIX Threads Producer-Consumer and can be found inside folder screenshots/producer_consumer_pthreads.

Figure 1.1. *Execution of the POSIX Threads Producer-Consumer Implementation*

```
violazeneli@group22-01:~/Paper2/code$ gcc producer_consumer_pthreads.c -o pc -pt
hread
violazeneli@group22-01:~/Paper2/code$ ./pc
Produced: 100000
Consumed: 100000
Execution Time: 0.055680 seconds
```

Figure 1.2. *Buffer Scalability Evaluation of the POSIX Threads Producer-Consumer Implementation*

```
violazeneli@group22-01:~/Paper2$ gcc code/pc_pthreads.c -o pc -pthread
violazeneli@group22-01:~/Paper2$ ./pc 10 100000 2 2
./pc 50 100000 2 2
./pc 100 100000 2 2
./pc 500 100000 2 2
N=10 M=2 K=2 ITEMS=100000 TIME=0.944661
N=50 M=2 K=2 ITEMS=100000 TIME=0.301538
N=100 M=2 K=2 ITEMS=100000 TIME=0.227865
N=500 M=2 K=2 ITEMS=100000 TIME=0.041624
```

Figure 1.3. *Stress-Test Execution of the POSIX Threads Producer-Consumer Implementation*

```
violazeneli@group22-01:~/Paper2/code$ gcc producer_consumer_pthreads.c -o pc -pthread
violazeneli@group22-01:~/Paper2/code$ ./pc
Produced: 100000
Consumed: 100000
Execution Time: 0.112021 seconds
```

Figures 2.1 – 2.3 are screenshots for script Java Producer-Consumer and can be found inside folder screenshots/producer_consumer_java.

Figure 2.1. *Execution of the Java Producer-Consumer Implementation*

```
violazeneli@group22-01:~/Paper2/code$ ./dp 5 1000 1
./dp 50 1000 1
N=5 ROUNDS=1000 MODE=1 TIME=1.784 sec
N=50 ROUNDS=1000 MODE=1 TIME=10.539 sec
```

Figure 2.2. *Buffer Scalability Evaluation of the Java Producer-Consumer Implementation*

```
violazeneli@group22-01:~/Paper2/code$ javac ProducerConsumer.java
violazeneli@group22-01:~/Paper2/code$ java ProducerConsumer 10 100 2 2
N=10 M=2 K=2 ITEMS=100 TIME=0.01785695
violazeneli@group22-01:~/Paper2/code$ java ProducerConsumer 10 1000 2 2
N=10 M=2 K=2 ITEMS=1000 TIME=0.045670491
violazeneli@group22-01:~/Paper2/code$ java ProducerConsumer 10 100000 2 2
N=10 M=2 K=2 ITEMS=100000 TIME=0.4154963
```

Figure 2.3. *Stress-Test Execution of the Java Producer-Consumer Implementation*

```
violazeneli@group22-01:~/Paper2/code$ javac ProducerConsumerJava.java
violazeneli@group22-01:~/Paper2/code$ java ProducerConsumerJava
Produced: 100000
Consumed: 100000
Execution Time: 0.132279193 seconds
```

Figures 3.1 – 3.3 are screenshots for script Semaphore-Based Dining Philosophers and can be found inside folder screenshots/dining_semaphore.

Figure 3.1. *Execution of the Semaphore-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2/code$ ./dp_sem 5 1000 1
Semaphore done
```

Figure 3.2. *Scalability Testing of the Semaphore-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2$ > results/semaphore.txt

for N in 5 10 20 50 100; do
    /usr/bin/time -v ./code/dp_sem $N 200 1 >> results/semaphore.txt 2>&
    echo "N=$N DONE ----------------" >> results/semaphore.txt
done
```

Figure 3.3. *Performance Metrics of the Semaphore-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2$ /usr/bin/time -v ./code/dp_sem $N 200 1
Semaphore done
        Command being timed: "./code/dp_sem 100 200 1"
        User time (seconds): 0.03
        System time (seconds): 0.49
        Percent of CPU this job got: 138%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.38
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 2532
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 285
        Voluntary context switches: 39739
        Involuntary context switches: 1765
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 4.1 – 4.3 are screenshots for script Monitor-Based Dining Philosophers and can be found inside folder screenshots/dining_monitor.

Figure 4.1. *Execution of the Monitor-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2/code$ ./dp_mon 5 1000
./dp_mon 10 1000
./dp_mon 50 1000
Monitor done
Monitor done
Monitor done
```

Figure 4.2. *Scalability Testing of the Monitor-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2$ > results/monitor.txt

for N in 5 10 20 50 100; do
    echo "RUNNING N=$N MONITOR"
    /usr/bin/time -v ./code/dp_mon $N 200 >> results/monitor.txt 2>&1
    echo "------------------------" >> results/monitor.txt
done
RUNNING N=5 MONITOR
RUNNING N=10 MONITOR
RUNNING N=20 MONITOR
RUNNING N=50 MONITOR
RUNNING N=100 MONITOR
```

Figure 4.3. *Performance Metrics of the Monitor-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2$ /usr/bin/time -v ./code/dp_mon 5 200
Monitor done
        Command being timed: "./code/dp_mon 5 200"
        User time (seconds): 0.00
        System time (seconds): 0.10
        Percent of CPU this job got: 59%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.19
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 1892
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 87
        Voluntary context switches: 3133
        Involuntary context switches: 319
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Figures 5.1 – 5.3 are screenshots for script Hierarchy-Based Dining Philosophers and can be found inside folder screenshots/dining_hierarchy.

Figure 5.1. *Execution of the Hierarchy-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2/code$ ./dp_hier 5 1000
./dp_hier 10 1000
Hierarchy done
Hierarchy done
```

Figure 5.2. *Scalability Testing of the Hierarchy-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2$ > results/hierarchy.txt

for N in 5 10 20 50 100; do
    echo "RUNNING N=$N HIERARCHY"
    /usr/bin/time -v ./code/dp_hier $N 200 >> results/hierarchy.txt 2>&1
    echo "------------------------" >> results/hierarchy.txt
done
RUNNING N=5 HIERARCHY
RUNNING N=10 HIERARCHY
RUNNING N=20 HIERARCHY
RUNNING N=50 HIERARCHY
RUNNING N=100 HIERARCHY
```

Figure 5.3. *Performance Metrics of the Hierarchy-Based Dining Philosophers Implementation*

```
violazeneli@group22-01:~/Paper2$ /usr/bin/time -v ./code/dp_hier 5 200
Hierarchy done
        Command being timed: "./code/dp_hier 5 200"
        User time (seconds): 0.00
        System time (seconds): 0.10
        Percent of CPU this job got: 31%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.31
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 1892
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 85
        Voluntary context switches: 2070
        Involuntary context switches: 373
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```